Solution Architect Case Study: One Customer, Many Systems

Thank you for taking the time to work on this. It's modeled on a real class of problem, but the company, systems and every record in the data are fictional.

Plan for about **6 to 8 hours**. We'd rather see a focused piece of work that runs than a complete design that doesn't. If you run out of time, tell us what you'd do next and why.

The company

Lumen Health sells at-home test kits and supplements to consumers.

- **Buying.** Customers buy online through an e-commerce platform, or by phone through sales reps working in a CRM.
- **Kits.** Every kit ships with a barcode. The customer registers it in the member portal before mailing a sample to a lab partner.
- **Results.** The lab returns results, which are filed in the electronic health record (EHR). The customer views them in the portal.
- **Marketing.** Marketing wants to nurture every customer by stage: *bought*, *registered*, *results ready*. Each stage gets a different message.

Each system has its own idea of who the customer is:

System	Identifies a customer by
E-commerce	A customer ID per email address. A new email creates a new customer
CRM	An account, matched by an older sync job (see the codebase)
Member portal	The email and details entered at registration
EHR	A chart, created from the registrant's name and date of birth

What people are seeing

The operations team has reported these symptoms. Nobody has diagnosed them yet:

1. Purchases sometimes show up on a family member's CRM account instead of the buyer's.
2. Customers' email addresses in the CRM change unexpectedly.
3. Some kit registrations are rejected as a "possible duplicate."
4. Some gift registrations fail, and support can't move the purchase to the recipient.
5. Marketing has sent "please register your kit" to customers who already have results.
6. Customers who opted out of texts start receiving them again after duplicate accounts are merged.

What we're giving you

This brief comes with a zip file. Unzip it and you'll find:

```
data/          Synthetic data from all four systems (about 2,000 customer records)
legacy-sync/   The current sync service (Python), including its git history
```

Data files

File	What it is
`ecom_customers.csv`	E-commerce customers
`ecom_orders.csv`	E-commerce orders, including gifts (`is_gift`)
`kits.csv`	Kit barcodes, linked to orders
`crm_accounts.csv`	CRM person accounts, including owner and consent fields
`crm_opportunities.csv`	CRM opportunities created by the order sync
`portal_registrations.csv`	Kit registrations from the member portal
`ehr_patients.csv`	EHR charts
`lab_results.csv`	Lab results, linked to kit and chart

Assume this data contains **protected health information** and design accordingly.

The codebase. `legacy-sync/` is the service that pushes orders and registrations into the CRM. Its documentation is deliberately thin, as it often is in practice. Expect to find your way around it yourself. Running it:

```
cd legacy-sync
python -m legacy_sync.replay --data ../data
python -m pytest
```

What we're asking for

1. Understand the current system

Explain how orders and registrations flow through `legacy-sync` today. For as many of the six symptoms as you can, identify the root cause, citing files and line numbers. Tell us what you'd change, and what you'd leave alone for now.

2. Target architecture

Design a single customer identity across these systems. Cover: - The data model - How records are matched, and how you handle uncertainty - How each system uses the identity - How it treats protected health information

3. A working slice

Write code that reads `data/` and produces one identity per real person. It must run with a single command, documented in your README, and write two files:

`identities.csv`, one row per customer record:

Column	Meaning
`system`	`ecommerce`, `crm`, `portal` or `ehr`
`record_id`	`customer_id`, `account_id`, `registration_id` or `chart_id`
`person_key`	Your identity key. Records with the same key are the same person
`confidence`	Your confidence in the record's link to that person, from 0 to 1

`review_queue.csv`: the links you weren't confident enough to make automatically, with the reason for each.

Use any language and any tools.

4. A build specification

Pick one component of your design and specify it precisely enough that a developer or an AI coding agent could build it without asking you questions.

5. A 90-day rollout

How you'd introduce this while orders and registrations keep flowing, and while the CRM is gradually replaced. One constraint is fixed: **a customer's kit registration must never be blocked waiting on identity.**

6. A one-page summary

For a non-technical executive: the problem, what you recommend and what it changes for the business.

How you use AI

We expect you to use AI tools: coding agents, chat assistants, whatever you normally use. We're interested in how you work with them, not whether you do. Include an `AI_LOG.md` with:

- Which tools you used, and for what
- Two or three moments where AI meaningfully sped you up
- At least two moments where AI was wrong or misleading, and how you caught it
- Your prompt history or session transcripts, if you can export them (raw is fine)

Submitting

Send a git repository or a zip with your code, `identities.csv`, `review_queue.csv`, `AI_LOG.md`, and your written answers in whatever format you prefer.

The review session (about 90 minutes)

1. **You present your work** (about 30 minutes).
2. **Live navigation** (about 20 minutes). We'll ask you to find and explain something in the codebase you may not have looked at yet, with your usual AI tools on a shared screen.
3. **A changed requirement** (about 20 minutes). We'll change one assumption and talk through how your design adapts.
4. **Questions** (about 20 minutes).

If anything in this brief is unclear, make a reasonable assumption, write it down, and keep going. That's part of what we're looking for.